

**2025-2026 SPRING
CME 2204 ALGORITHM ANALYSIS**

**Assignment-1
Comparison of Tim Sort and Quick Sort**

**Due
15.04.2026 23:55**

In this assignment, you will implement and test two state-of-the-art sorting algorithms: Tim sort and Quick sort. You will implement your own algorithms and compare their performance on 1000000 random, semi-ordered, increasing-order, and decreasing-order integers.

1. Algorithms

1.1. Tim Sort

Tim Sort is a hybrid, stable sorting algorithm derived from Merge Sort and Insertion Sort. It was designed by Tim Peters in 2002 for the Python programming language and is now the default sorting algorithm for Python, Java (`Arrays.sort`), and Android. It is specifically optimized to perform exceptionally well on real-world data, which often contains pre-existing ordered sequences. First, the array is broken down into smaller chunks of data known as runs. These runs are sorted using insertion sort, then merged using merge sort.

Step 1: The array is broken down into runs.

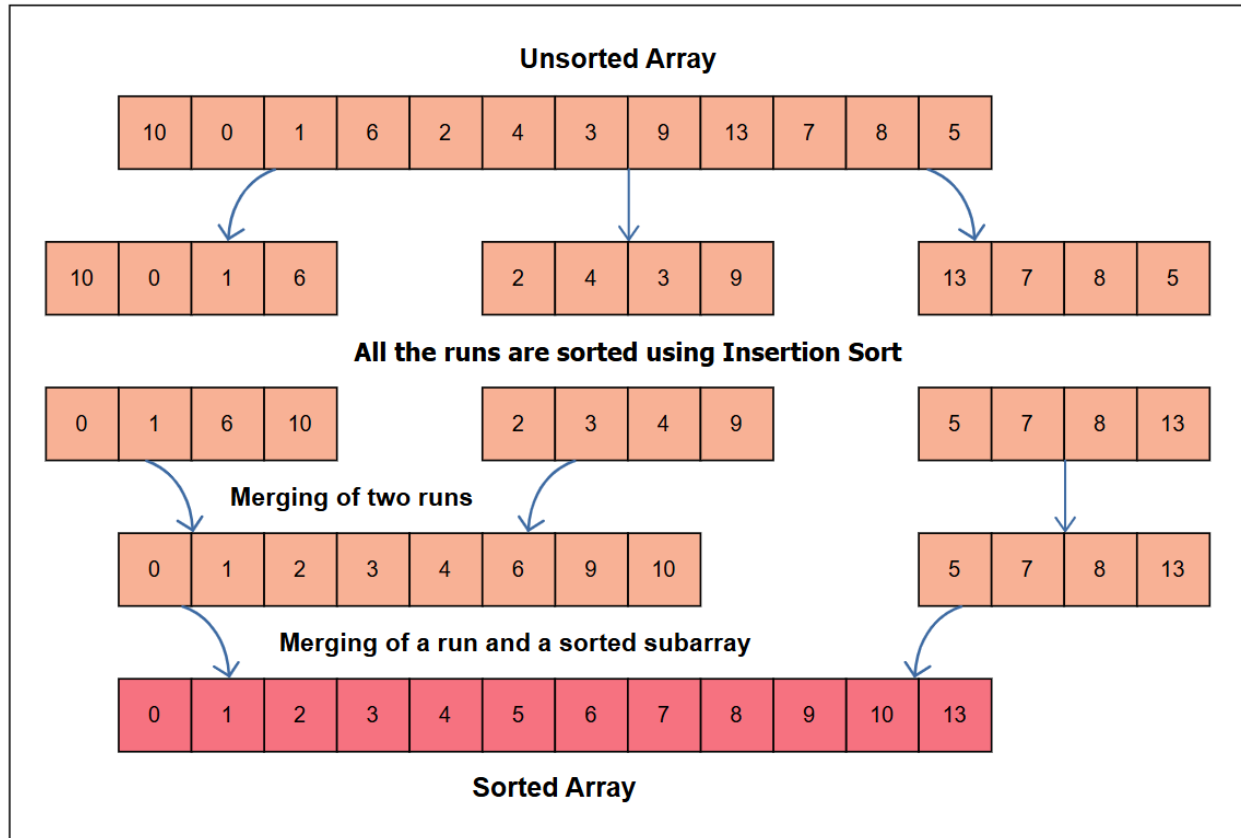
Preferably, the size of the run (minrun) should be such that (n/minrun) is a power of 2 (since merge sort gives optimal results in that case).

Step 2: Initialize the size of the run as 32 or 64.

Step 3: Sort every run using insertion sort. If the array size is smaller than 64, then it is just sorted using insertion sort.

Step 4: Merge all the runs one by one using merge sort.

Step 5: After every iteration, double the size of the merged sub-array.



1.2. Quick Sort

Quick sort is a highly efficient divide-and-conquer sorting algorithm. It works by selecting a 'pivot' element from the list and partitioning the other elements into two sub-arrays: those less than the pivot and those greater than the pivot. The pivot element is then in its final sorted position. The algorithm then recursively sorts the sub-arrays.

In the quick sort algorithm, the pivot can be selected with one of the following strategies:

- First element
- Last element
- Middle element
- Random Element
- Median Element

2. Input Files

You are given two input files containing 1000000 random integer numbers.

- random.txt
- semi_ordered.txt

These files contain a number in each line. Place them in your project directory for direct access. They can be replaced with new ones during code control.

You must create arrays for the increasing-order and decreasing-order integers in your code.

3. Output Files

After the algorithms are executed, the results must be written to the output files in the same location as the input files (a total of 24 output files). Name the output files with the given name convention below:

(algorithm)_(data_type)_out.txt

Example = timsort_random_out.txt

algorithm=

timsort/quicksort_first/quicksort_last/quicksort_middle/quicksort_random/quicksort_median

data_type = random/semi_ordered/increasing/decreasing

4. Submission

You must upload your Java source code and your report via **online.deu.edu.tr**.

- Name your source code as (student_number)_sources.zip (exp. 2043901815_sources.zip). Do not upload the input or output files.
- Name your report as (student_number)_report.pdf (exp. 2043901815_report.pdf).

You must prepare a project report that

- containing a cover page specifying your name and student number,
- describing your Java code,
- indicating the hardware specifications of the machine on which you are running the algorithms,
- presenting the performance comparison matrix of the sorting algorithms (Table 1),
- including a discussion about the results.

You are expected to compare each sorting algorithm's running time (in milliseconds) for different inputs and fill out Table 1 accordingly. After you fill out the table, discuss the best and worst algorithms for various conditions.

* Note that the elapsed time of creating the arrays and writing output files should not be considered when calculating the total running time of sorting algorithms. You only need to check how long the actual sorting algorithm is running.

Table 1. Performance comparison matrix (in ms)

		RANDOM INTEGERS (Read numbers from input files)	SEMI-ORDERED INTEGERS (Read numbers from input files)	INCREASING INTEGERS (Create arrays in your code)	DECREASING INTEGERS (Create arrays in your code)
Tim Sort					
Quick Sort	First element				
	Last element				
	Middle element				
	Random Element				
	Median Element				

5. Plagiarism Control

The assignment will be submitted individually. The submissions will be checked for code similarity. Copy assignments will be graded as zero!

6. Grading Policy

Job	Grade %
Tim sort	30
Quick sort	30
Reading input files and generating output files	10
Report	30